

Project: Treasure Hunt Game

Phase: Weeks 8–10 (Cross-Platform, Polish, Testing & Submission)

1. Objective

This final phase brings the Treasure Hunt game to completion by adding cross-platform touch controls, file persistence for leaderboards/settings, rigorous error handling and bug fixing, and comprehensive documentation for final submission.

2. Detailed Task Breakdown

2.1 Week 8: Cross-Platform Support & UX Polish

- On-Screen & Touch Controls: Implement a responsive virtual control overlay (directional pad / action buttons) on Java Swing panels to support touch-screen and nonkeyboard environments.
- Data Persistence via File I/O:
 - o Create a `Storage.java` utility class handling object serialization or file-based text storage.
 - o Persist player settings (audio, control preferences) and the high-score leaderboard across application restarts.
- UI & Menu Polish: Refine main menus, pause overlays, victory/game-over screens, and HUD elements with consistent styling and smooth rendering.

2.2 Week 9: Error Handling & Testing

- Procedural Generation Validation: Test map generation algorithms across multiple consecutive runs to ensure rooms are always solvable and items/doors never spawn outof-bounds or get blocked.
- Edge Case Mitigation: Implement robust checks for critical gameplay states:
 - o Torch/battery resource reaching zero.
 - o Invalid boundary or collision movements.
 - o Interactions with empty inventories or missing prerequisite items.

- ❑ Performance Optimization: Fix rendering bottlenecks, eliminate memory leaks in custom paint loops, and optimize frame-rate stability.

2.3 Week 10: Final Testing, Documentation & Submission

- ❑ Full System Playthrough: Conduct end-to-end integration testing across different screen resolutions and platforms (desktop/touch).
- ❑ Documentation (docs/): Prepare a clean, professional README.md featuring setup instructions, controls, architectural design notes, and screenshots.
- ❑ Final Packaging & Submission: Clean up source code, verify all class encapsulations and package structures, and prepare the final deliverable archive.

3. Deliverables

- ❑ High scores and game settings successfully persisted using Java Serialization (Storage.java).
- ❑ Polished menus and responsive HUD overlays.
- ❑ Comprehensive edge-case handling (torch depletion, invalid movement, procedural bounds).
- ❑ Complete project documentation (docs/README.md with setup notes and screenshots).
- ❑ Fully tested, submission-ready codebase.

4. Tools & Resources

- ❑ Java JDK 11+
- ❑ IntelliJ IDEA / VS Code
- ❑ Java Swing & AWT (Graphics2D, Event Listeners)
- ❑ Java I/O & Serialization (Serializable, ObjectOutputStream, ObjectInputStream)